base.py — technical

`src/moe_pruning/tasks/base.py · 88 lines · moe_pruning.tasks.base`

Purpose

Abstract task interface; see seq2seq_denoising.py and causal_lm.py for implementations.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class TaskObjective`

lines 22-79 · bases ABC

Interface: how a model family's data becomes inputs and how loss is computed.

`TaskObjective.tokenize_calibration`

```
def tokenize_calibration(self, records: list[CalibrationRecord], tokenizer: PreTrainedTokenizerBase, max_source_length: int, max_target_length: int) -> list[TokenizedPair]
```

lines 36-43

`human` is the input/prompt, `response` the teacher-forced target.

`TaskObjective.collate`

```
def collate(self, items: Sequence, pad_token_id: int) -> ModelBatch
```

lines 49-50

Right-padded batch with per-stack real-token masks.

`TaskObjective.iter_batches`

```
def iter_batches(self, items: Sequence, batch_size: int, pad_token_id: int) -> Iterator[ModelBatch]
```

lines 52-54

Split items into collated batches.

`TaskObjective.eval_settings`

```
def eval_settings(self, cfg) -> dict
```

lines 60-61

Task-specific settings that determine the evaluation examples.

`TaskObjective.build_eval_example`

```
def build_eval_example(self, example_id: int, text: str, tokenizer: PreTrainedTokenizerBase, cfg)
```

lines 64-65

One evaluation example (must expose `example_id`, `input_ids`, `labels`).

`TaskObjective.token_nll`

```
def token_nll(self, logits: torch.Tensor, labels: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]
```

lines 68-69

Per-example (sum of token NLL, number of scored tokens).

`TaskObjective.predictions`

```
def predictions(self, logits: torch.Tensor, labels: torch.Tensor) -> list[list[int]]
```

lines 72-73

Per-example argmax token at every scored position (same order as `token_nll`).

`TaskObjective.qualitative`

```
def qualitative(self, model: nn.Module, tokenizer: PreTrainedTokenizerBase, examples: list, device:
torch.device, cfg) -> list[dict]
```

lines 76-79

`[{"example_id", "inputs": {label: text}, "prediction"}]` from greedy generation.

`clean_decode`

```
def clean_decode(tokenizer: PreTrainedTokenizerBase, ids: list[int]) -> str
```

lines 82-88

Decode keeping special tokens such as T5 sentinels, but without pad/EOS noise.

1. Decode with special tokens kept, remove pad/EOS strings, collapse whitespace.